```
print("Experiment 6")
print("Parth Mahesh Madrewar")
print("Reg. No. : 241080036")
```

```
Experiment 6
Parth Mahesh Madrewar
Reg. No. : 241080036
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc


data = pd.read_csv("/content/mushrooms.csv")
print(data.head())

X = data.drop("class", axis=1)
y = data["class"]

X = pd.get_dummies(X, drop_first=True)
X = X.astype(float)

y = y.map({'e': 0, 'p': 1})
y = y.astype(float)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
  class cap-shape cap-surface cap-color bruises odor gill-attachment  \
0     p         x           s         n       t    p               f
1     e         x           s         y       t    a               f
2     e         b           s         w       t    l               f
3     p         x           y         w       t    p               f
4     e         x           s         g       f    n               f

  gill-spacing gill-size gill-color  ... stalk-surface-below-ring  \
0            c         n          k  ...                        s
1            c         b          k  ...                        s
2            c         b          n  ...                        s
3            c         n          n  ...                        s
4            w         b          k  ...                        s

  stalk-color-above-ring stalk-color-below-ring veil-type veil-color  \
0                      w                      w         p          w
1                      w                      w         p          w
2                      w                      w         p          w
3                      w                      w         p          w
4                      w                      w         p          w

  ring-number ring-type spore-print-color population habitat
0           o         p                 k          s       u
1           o         p                 n          n       g
2           o         p                 n          n       m
3           o         p                 k          s       u
4           o         e                 n          a       g

[5 rows x 23 columns]
```

```
class LogisticRegressionScratch:
    def __init__(self, learning_rate=0.01, epochs=1000):
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = 0
        self.loss = []

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def fit(self, X, y):
```

```
        rows, columns = X.shape

        self.weights = np.zeros(columns)
        self.bias = 0
        for i in range(self.epochs):
            z = np.dot(X, self.weights) + self.bias
            y_pred = self.sigmoid(z)
            error = y_pred - y

            epsilon = 1e-10
            bce = -np.mean(y * np.log(y_pred + epsilon) + (1 - y) * np.log(1 - y_pred + epsilon))
            self.loss.append(bce)          # -> also known as loss function

            weight_gradient = np.dot(X.T, error) / rows
            bias_gradient = np.mean(error)

            self.weights = self.weights - self.learning_rate * weight_gradient
            self.bias = self.bias - self.learning_rate * bias_gradient

    def predict_proba(self, X):
        return self.sigmoid(np.dot(X, self.weights) + self.bias)

    def predict(self, X):
        return (self.predict_proba(X) >= 0.5).astype(int)
```

```
model = LogisticRegressionScratch(learning_rate=0.1, epochs=1000)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("\nBias:")
print(model.bias)
print("\nWeights:")
print(model.weights)

# Performance
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\nScratch Logistic Regression")
print("Accuracy :", acc)
print("Confusion Matrix:\n", cm)

# Plot training loss
plt.plot(model.loss)
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Binary Cross-Entropy")
plt.grid()
plt.show()

fpr, tpr, thresholds = roc_curve(y_test, y_pred)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {roc_auc:.2f})")
plt.plot([0, 1], [0, 1], linestyle="--")

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve")
plt.legend()
plt.show()

# Logistic Regression using Scikit-Learn
model2 = LogisticRegression(max_iter=1000)
model2.fit(X_train, y_train)
y_pred2 = model2.predict(X_test)

# Performance
acc2 = accuracy_score(y_test, y_pred2)
cm2 = confusion_matrix(y_test, y_pred2)

print("\nScikit-Learn Logistic Regression")
print("Accuracy :", acc2)
print("Confusion Matrix:\n", cm2)
```